

Human Activity Recognition (HAR) on Sensor Data

Team Members: Anjali Barvaliya, Parth Ravaliya, Prajnya Kalmene, Vishal Chitroda

Agenda

01

Overview & Motivation

03

Project goals and success criteria

05

Data pipeline (Data Loading and Preprocessing)

07

Baseline model and what it establishes

09

Results and interpretation

02

Problem & Constraints

04

Dataset Understanding

06

Windowing and evaluation protocol

08

Deep learning models and efficiency-first design

10

Project Value & Conclusion

Context and Motivation

01

Why smartphone HAR matters

High-value applications

- Fitness & wellness tracking (steps, intensity, routines)
- Healthcare monitoring support (mobility patterns, adherence)
- Elderly care / safety (inactivity detection, fall-risk signals)
- Smart environments (context-aware automation)

Why phones (not wearables)

- Ubiquitous: users already carry phones
- Cost-effective: no extra devices
- Sensors available: accelerometer (and often gyro)
- Challenge: placement variability + power constraints



What we built, how we evaluated it, and what we achieved

What we built, how we evaluated it, and what we achieved

Core system

Input: phone accelerometer time-series (ax, ay, az)
Output: activity label + confidence
Deployment: TFLite INT8 model (edge-friendly)
UX: hybrid explanations (rule-based fallback + FLAN-T5 when confident)

Realistic evaluation

Subject-independent split (test users never seen in training)
Leakage prevention: split by subject first, then window
Metrics: Accuracy + Macro-F1 + per-class breakdown

18-class (accel-only)

Acc ~0.24 • Macro-F1 0.23

MVP 5-class

Acc 0.77 • Macro-F1 0.77

Deployment

TFLite INT8 + hybrid LLM

Key takeaway:

accel-only is strong for locomotion/posture; fine-grained daily actions are challenging without gyro/watch.
We present both 18-class realism and a deployable 5-class MVP.

Problem Statement

02

Recognize activities with realistic constraints

Problem

Given raw phone accelerometer time-series, classify the user's activity in short windows (e.g., walking, jogging, stairs, sitting, standing). Constraints: subject-independent generalization, phone placement variability, and on-device compute/battery limits.

Engineering constraints we enforce

- No wearables required (phone-only)
- Small models (edge-friendly)
- Avoid data leakage (split-by-subject)
- Honest metrics (macro-F1 + per-class)

Practical UX constraints

- Provide confidence / uncertainty messaging
- Explain predictions in plain language
- Choose MVP classes that are reliable with accel-only

Project Goals and Success Criteria

03

What “success” means for this project

Goals

- 1) Build an end-to-end pipeline from CSV → windows → model → predictions
- 2) Compare baseline ML vs deep learning sequence models
- 3) Evaluate on unseen users (subject-independent)
- 4) Deployable model (TFLite INT8) + explanations

Success criteria

- Correct evaluation: no leakage, train-only normalization
- Metrics beyond accuracy: macro-F1 + per-class report
- MVP performance acceptable for demo and deployment
- Clear interpretability + professional reporting

Dataset Understanding

04

WISDM-51 dataset (what it contains)

Dataset overview

- 51 subjects
- 18 activities (locomotion, hand-based daily living, eating activities)
- Sensors: phone accelerometer/gyroscope, watch accelerometer/gyroscope
- Each activity recorded for ~3 minutes per subject

Our project choice

- Use phone accelerometer only (ax, ay, az)
- Reason: cost + energy efficiency; no wearables
- Trade-off: fine-grained activities become harder
- Mitigation: present both 18-class realism + 5-class MVP

Dataset Understanding

File format & sampling characteristics (from sample subject file)

Columns (accelerometer phone CSV)

Subject-id • Activity Label • Timestamp • X-accel • Y-accel • Z-accel
Each row is one sensor reading at a timestamp.

```
1 # Example row (Subject-id, Activity, Timestamp, X, Y, Z)
2 1600, A, 252207666810782, -0.364761, 8.793503, 1.055084
```

Sampling rate (estimated)

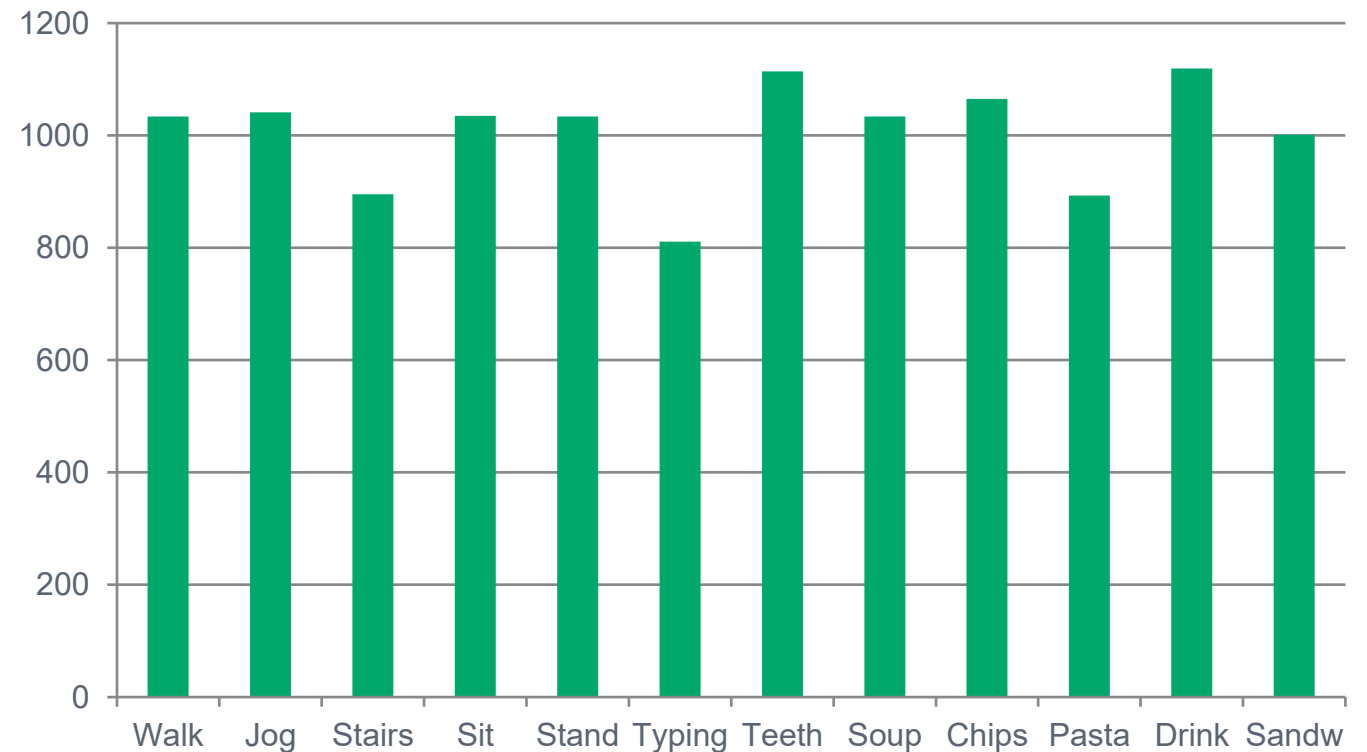
Median timestamp $\approx 50,354,004$ ns ≈ 0.050 s $\rightarrow$ ~ 20 Hz. Large timestamp gaps occur; we split segments on big gaps before windowing.

Dataset Understanding

Activity label space (18 activities) + test-set support

Code → Activity

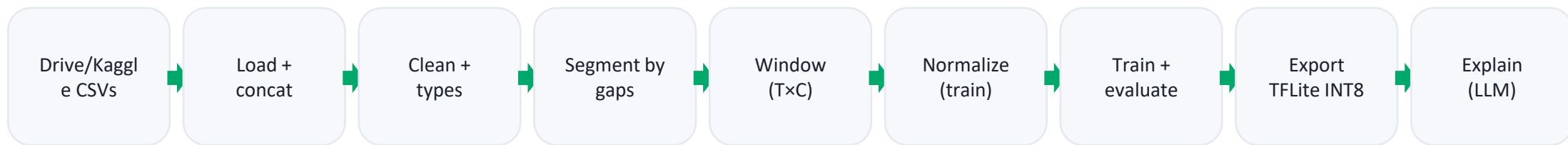
A Walking
B Jogging
C Stairs
D Sitting
E Standing
F Typing
G Brushing Teeth
H Eating Soup
I Eating Chips
J Eating Pasta
K Drinking (Cup)
L Eating Sandwich
M Kicking (Soccer)
O Playing Catch
P Dribbling (Basketball)
Q Writing
R Clapping
S Folding Clothes



Data pipeline

05

From raw CSV to deployable model



Implementation notes (from code)

- CSV schema normalized (subject/activity/timestamp/ax/ay/az)
- Segment breaks: split sequences where timestamp gaps are large
- Windows: size 128, stride 64 for better sequence context (MVP)
- Train-only normalization to avoid leakage
- Early stopping + LR scheduling to stabilize training

Data loading (Google Drive → DataFrame)

Using Google Colab

What this code does

- Mount Drive and point to dataset folder
- Find all accel files (51 subjects)
- Read each CSV, rename columns, set types
- Concatenate into one DataFrame

```
1 from google.colab import drive
2 drive.mount('/content/drive')
3
4 DATA_DIR = "/content/drive/MyDrive/HAR_WISDM/WISDM-51"
5
6 accel_files = sorted(glob.glob(os.path.join(DATA_DIR,
7 "*accel_phone*.csv")))
8
9 def read_accel_file(path):
10     df = pd.read_csv(path)
11     df = df.rename(columns={
12         "Subject-id": "subject",
13         "Activity Label": "activity",
14         "Timestamp": "timestamp",
15         "X-accel": "ax",
16         "Y-accel": "ay",
17         "Z-accel": "az"
18     })
19     return df
```

Data Preprocessing

Preprocessing & derived signals (cheap but effective)

Why derive magnitude + jerk?

- Orientation robustness: magnitude is less sensitive to phone rotation
- Intensity cue: locomotion vs static activities
- Jerk captures motion changes (helps stairs vs walking)
- Adds signal without new sensors or big compute

```
1 ax = seg["ax"].to_numpy(np.float32)
2 ay = seg["ay"].to_numpy(np.float32)
3 az = seg["az"].to_numpy(np.float32)
4
5 acc_mag = np.sqrt(ax*ax + ay*ay + az*az).astype(np.float32)
6 jerk = np.diff(acc_mag, prepend=acc_mag[0]).astype(np.float32)
```

Normalization (train-only)

We compute mean/std on training windows only and apply to train/test. This prevents leakage and stabilizes deep learning training.

Windowing and evaluation protocol

06

Windowing strategy (clean windows, no mixed labels)

Rules used in code

- 1) Group by (subject, activity) → windows contain one label
- 2) Sort by timestamp
- 3) Split segments on large timestamp gaps
- 4) Slide a fixed window: size=128, stride=64 (MVP)

```
1 jumps = g["timestamp"].diff().fillna(0) > gap_threshold
2 seg_id = jumps.cumsum()
3
4 for _, seg in g.groupby(seg_id):
5     if len(seg) < window_size: continue
6     data = build_channels(seg)
7     for start in range(0, n-window_size+1, stride):
8
9         X_list.append(data[start:start+window_size])
10        y_list.append(act)
```

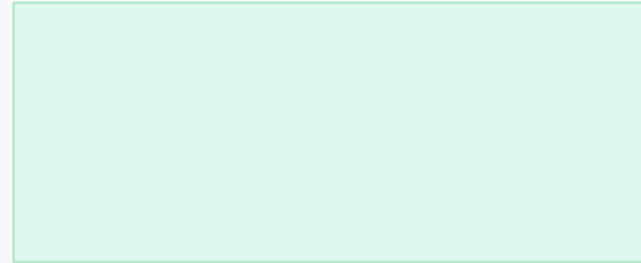
Windowing and evaluation protocol

Subject-independent evaluation (no user leakage)

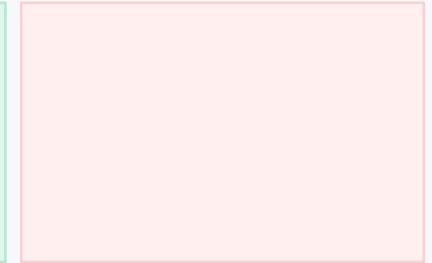
Protocol

- Split by subject (train users $\neq$ test users)
- Window only within each split
- Train-only normalization and label encoding
- Metrics: accuracy, macro-F1, per-class F1, confusion analysis

Train subjects (80%)



Unseen users



Why macro-F1?

Macro-F1 weights each class equally and reveals whether the model is ignoring difficult classes (especially in 18-class setting).

Baseline model and what it establishes

07

Random Forest baseline with GroupKFold (subject-independent)

Baseline pipeline

Window → feature extraction → Random Forest

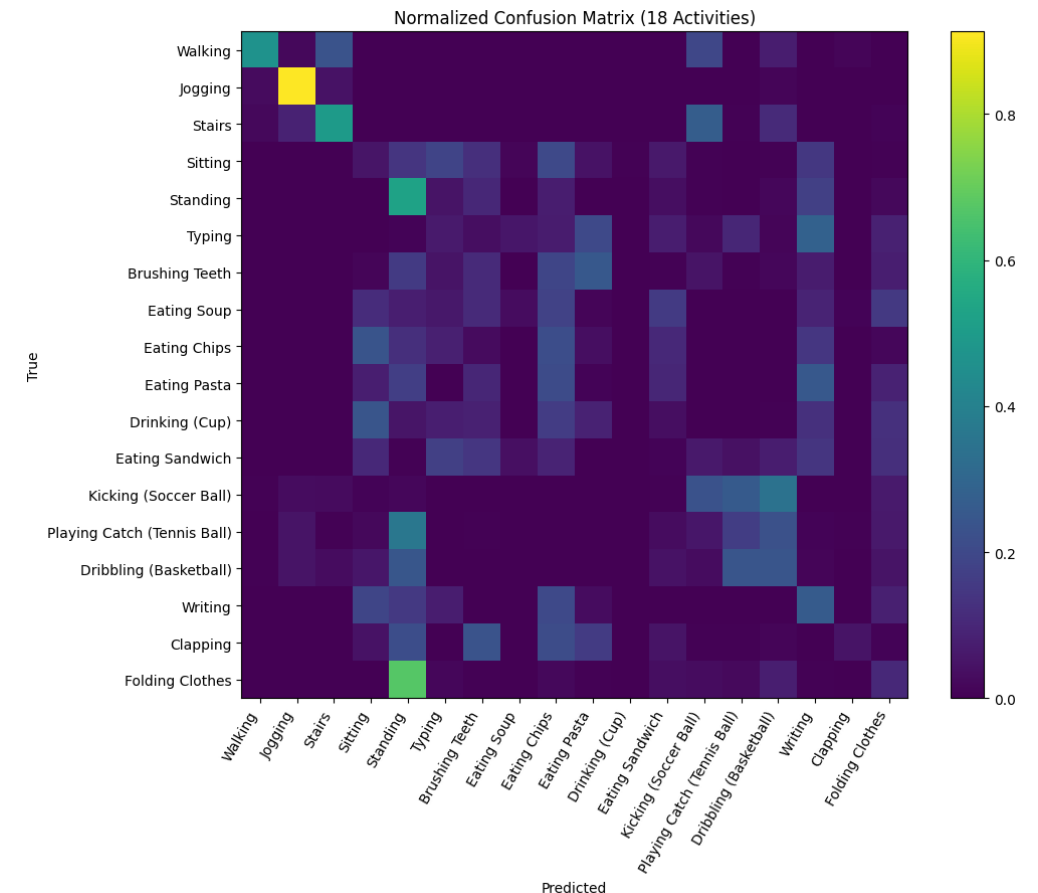
Features per channel (ax,ay,az + mag):

- mean, std, min, max
- median, energy

Why baseline matters:

- sanity-check preprocessing
- strong reference before deep learning

Accuracy : ~26% (for 18 activities)



Deep learning models (Tiny CNN / GRU) and efficiency-first design

08

Why small sequence models work for HAR

Model choices

Tiny CNN (1D):

- learns local temporal patterns (rhythm, bursts)
- efficient inference with separable convolutions

GRU (optional):

- models longer temporal dependencies
- generally cheaper than LSTM

Shared inputs:

- windowed sequences ($T \times C$)
- train-only normalization

Efficiency-first design

- Accelerometer-only sensing
- SeparableConv1D + GlobalAveragePooling
- Small parameter count
- Early stopping reduces training compute
- INT8 quantization for deployment

Deep learning models

Tiny CNN (MVP) + training safeguards

Architecture (high level)

Input (128×5) → Conv1D(24) → BN/ReLU → MaxPool
→ SepConv1D(48) → BN/ReLU → MaxPool
→ SepConv1D(64) → BN/ReLU
→ GlobalAvgPool → Dropout → Dense(softmax)

Training details (code)

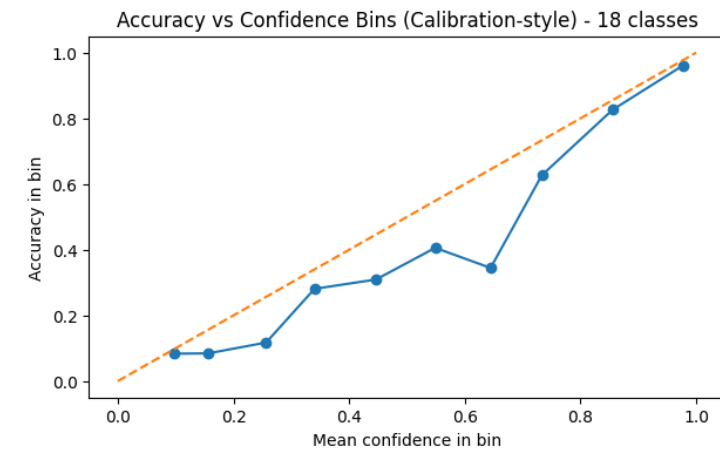
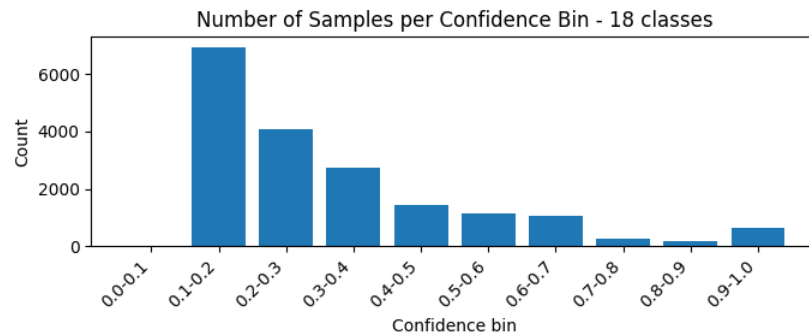
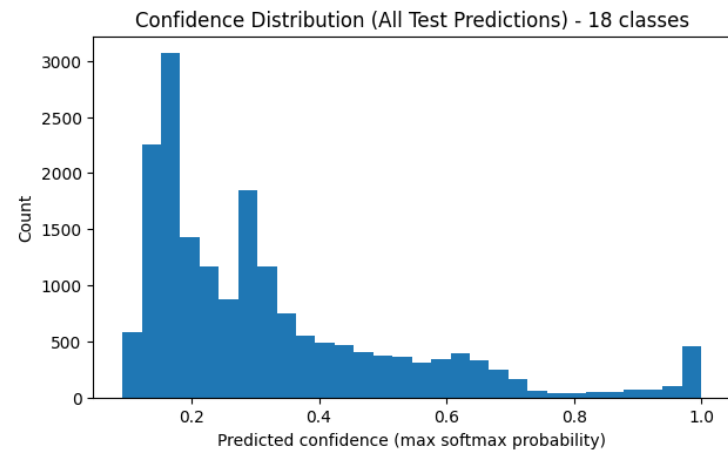
- EarlyStopping(patience=4)
- ReduceLROnPlateau for stability
- class_weight = balanced (helps minority classes)
- validation_split = 0.15

```
1 model = build_small_cnn((WINDOW_SIZE, C),  
2 num_classes)  
3  
4 model.compile(optimizer=Adam(1e-3),  
5               loss='sparse_categorical_crossentropy',  
6               metrics=['accuracy'])  
7  
8 callbacks = [  
9     EarlyStopping(monitor='val_loss',  
10                  patience=4, restore_best_weights=True),  
11     ReduceLROnPlateau(monitor='val_loss',  
12                      factor=0.5, patience=2, min_lr=1e-5)  
13 ]  
14  
15 model.fit(X_train_n, y_train_id,  
16          validation_split=0.15,  
17          class_weight=class_weights,  
18          epochs=20, batch_size=128,  
19          callbacks=callbacks)
```

Results and interpretation

09

18-class results (accel-only, subject-independent)



Results and interpretation

09

18-class results (accel-only, subject-independent)

Accuracy

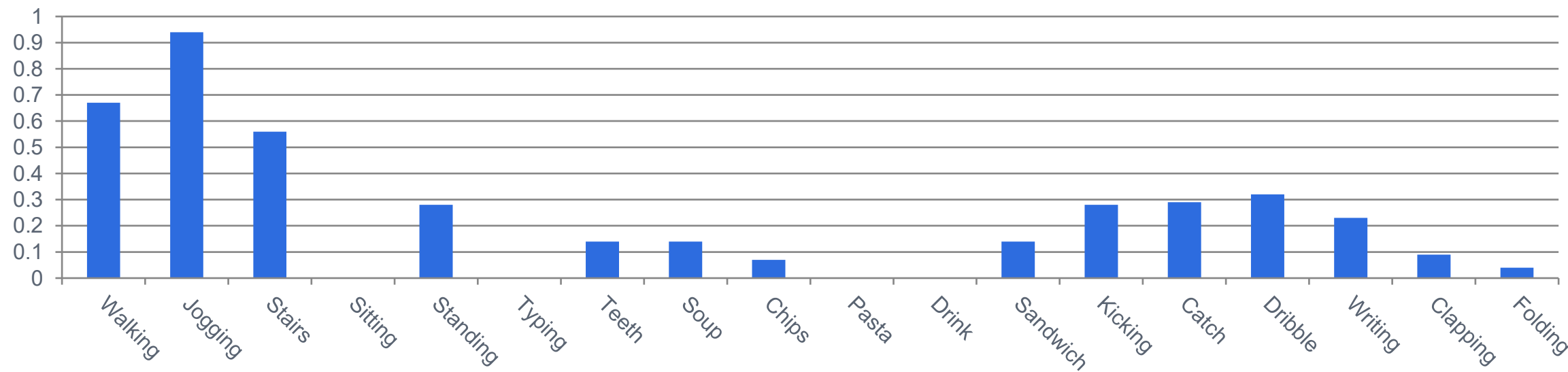
0.248

Macro-F1

0.233

Observation

Many classes ≈ 0



Results and interpretation

Why 18-class is difficult with accelerometer-only

Signal limitation

Many daily actions (eating, writing, brushing teeth) are subtle in phone accel, and are sensitive to: phone placement, user style, and short window context.

Result: model learns locomotion well but struggles on fine-grained actions.

What the metrics show

- High F1: Jogging, Walking, Stairs
- Near-zero recall: several hand/eating classes
- Macro-F1 reveals ignored classes even when accuracy seems OK

Conclusion: we present an MVP label set for deployment.

Two responsible choices

- 1) Keep 18-class as a realism experiment and report limitations honestly
- 2) Build a deployable MVP (5 classes) where accel-only is reliable

Results and interpretation

MVP 5-class results (deployable target)

Accuracy

0.695

Macro-F1

0.674

Classes

Walk/Jog/Stairs/Sit/Stand

==== MVP Test Results (Subject-independent) ====

Accuracy: 0.7763445128001588

Macro-F1 : 0.7711081797067558

Report:

	precision	recall	f1-score	support
A	0.74	0.88	0.80	1034
B	0.96	0.88	0.92	1041
C	0.82	0.56	0.66	895
D	0.78	0.69	0.73	1035
E	0.66	0.86	0.74	1034
accuracy			0.78	5039
macro avg	0.79	0.77	0.77	5039
weighted avg	0.79	0.78	0.77	5039

Interpretation

- Jogging + postures are robust under accel-only
- Walking recall is lower: model often predicts similar locomotion classes
- Next steps to improve walking: gravity/body separation, longer windows, mild overlap tuning

This MVP is suitable for a demo + on-device deployment.

Results and interpretation

Error analysis (MVP) and targeted improvements

Observed issue

Walking: high precision but low recall

Likely causes:

- Similar periodic motion to stairs
- Phone placement/orientation variability
- Window length may be borderline for slow walking patterns

Practical improvements (still low power)

- Add gravity/body separation (low-pass)
- Use slightly longer window for walking (e.g., 256) while keeping stride moderate
- Calibrate confidence threshold for safer UX
- Optional: personalize using a short per-user calibration session

```
1 # Gravity separation (cheap orientation
robustness)
2 gwin = 10
3 k = np.ones(gwin, dtype=np.float32) / gwin
4
5 gx = np.convolve(ax, k, mode='same')
6 # body accel = raw - gravity
7 bx = ax - gx
8 body_mag = np.sqrt(bx*bx + by*by + bz*bz)
9 jerk = np.diff(body_mag, prepend=body_mag[0])
10
11 # Use [ax,ay,az, body_mag, jerk] as input
channels
```

Project value & impact

10

Deployment: TFLite INT8 (edge-ready)

Why quantize to INT8?

- Smaller model (often ~4× smaller vs float32)
- Faster CPU inference on edge devices
- Lower memory bandwidth → often lower energy

We export the trained CNN to TFLite with full integer quantization.

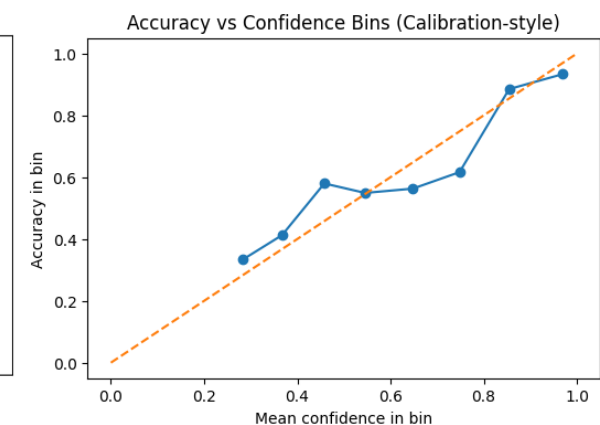
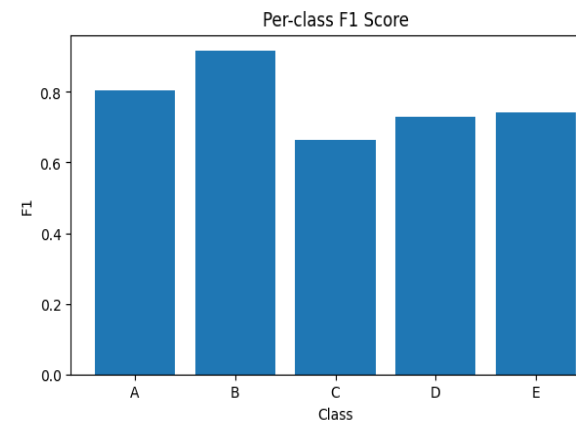
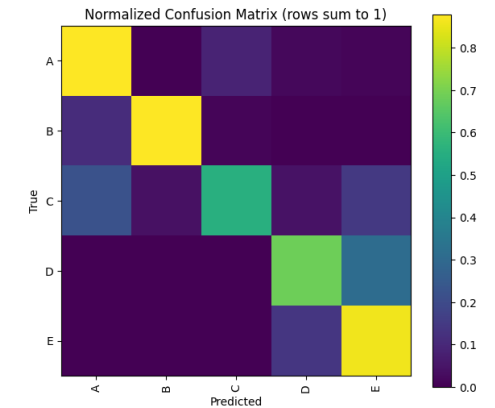
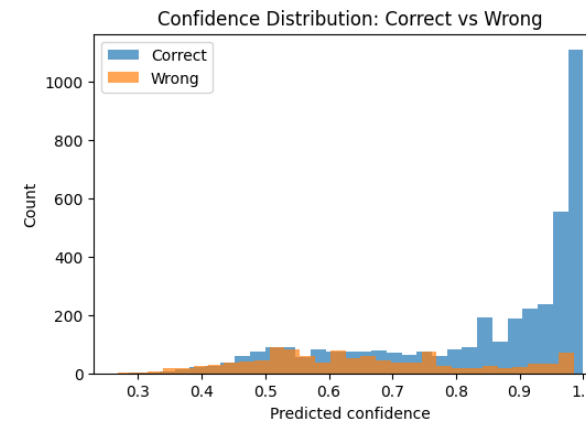
Low Computing Power or Cost Efficient

Accuracy

0.77

Macro-F1

0.771



Project value & impact

Efficiency check: benchmark inference cost (methodology)

What we measure

- Avg inference time per window (ms)
- Compare: Keras float32 vs TFLite INT8
- Warmup runs to avoid one-time overhead

Note: Colab timings $\neq$ phone timings, but relative speedup trends are useful.

```
1 # Keras timing
2 _ = model.predict(X_bench[:10])
3 t0 = time.perf_counter()
4 _ = model.predict(X_bench)
5 t1 = time.perf_counter()
6 keras_ms = (t1-t0)/N * 1000

7 for i in range(N):
8     x_q = quantize_to_int8(X_bench[i:i+1])
9     interpreter.set_tensor(in_idx, x_q)
10    interpreter.invoke()
```

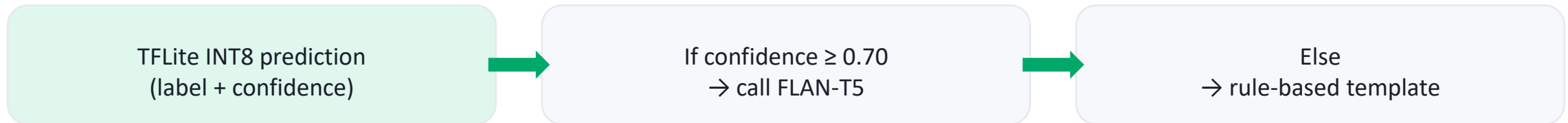
✓ Keras Float32 Model Timing
Total time (s): 10.4288
Avg per window (ms): 20.8575

✓ TFLite INT8 Model Timing
Total time (s): 0.0366
Avg per window (ms): 0.0732

✗ Comparison Summary
Keras avg (ms): 20.8575
TFLite avg (ms): 0.0732
Estimated speedup (x): 285.12

Project value & impact

Hybrid explanations: consistent fallback + FLAN-T5 when confident



Inputs to the explainer

Predicted activity • confidence • motion stats (mean/std magnitude, jerk).
Rules avoid medical claims and always show uncertainty when confidence is low.

Example output (style)

“Prediction: Jogging (confidence 0.92). Motion is high and rhythmic (higher jerk), consistent with running. Tip: keep the phone in a consistent pocket for stable readings.”

Low-confidence fallback adds: “Model is uncertain; phone placement may affect the signal.”

Conclusion & next steps

What we delivered

- Reproducible Colab pipeline: load → preprocess → window → train → evaluate
- Honest evaluation on unseen users (subject-independent)
- Deployable MVP model (TFLite INT8)
- Professional evaluation plots + hybrid explanation layer

Key lessons

- Accel-only is reliable for locomotion/posture
- Fine-grained daily actions need richer sensing or personalization
- Macro-F1 is essential for multi-class fairness
- Deployment constraints shape model design

Next steps

- Improve walking recall: gravity/body separation, longer windows
- Add gyro if allowed (still phone-only)
- On-device streaming + threshold-based decisions
- User-specific calibration to boost generalization

References

1. J. R. Kwapisz, G. M. Weiss, and S. A. Moore, "Activity Recognition Using Cell Phone Accelerometers," *ACM SIGKDD Explorations Newsletter*, vol. 12, no. 2, pp. 74–82, 2011.
2. G. M. Weiss, "WISDM Dataset Description (Smartphone & Smartwatch Sensor Data)," UCI Machine Learning Repository (dataset documentation PDF).
3. Kaggle dataset page (distribution copy used in your project): "WISDM-51" dataset description.
4. TensorFlow, "Post-training quantization (TensorFlow Model Optimization Guide)."
5. TensorFlow, "tf.lite.RepresentativeDataset documentation (calibration dataset for quantization)."
6. TensorFlow (Blog), "Post-Training Integer Quantization" (full-integer quantization workflow + representative dataset concept).
7. Google AI Edge / LiteRT docs, "Post-training integer quantization (full integer / INT8)."
8. A. G. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv:1704.04861, 2017. (Depthwise separable convs used for efficient models.)
9. J. Chung et al., "Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling," arXiv:1412.3555, 2014. (GRU reference.)
10. S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
11. E. B. Wilson, "Probable Inference, the Law of Succession, and Statistical Inference," *Journal of the American Statistical Association*, 1927. (Wilson score interval used for binomial accuracy CIs.)
12. scikit-learn documentation: classification_report (precision/recall/F1 reporting used in your results).
13. scikit-learn documentation: calibration_curve (reliability diagram / calibration plots).
14. H. W. Chung et al., "Scaling Instruction-Finetuned Language Models," arXiv:2210.11416 (FLAN / instruction tuning basis for Flan-T5-style explainers).
15. C. Raffel et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *JMLR*, 2020. (T5 foundation model.)
16. J. Wei et al., "Finetuned Language Models Are Zero-Shot Learners," arXiv:2109.01652, 2021. (Instruction tuning concept, FLAN roots.)
17. N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," arXiv:1908.10084, 2019. (Embedding methodology behind sentence-transformers usage.)



Thank You

Team Members: Anjali Barvaliya, Parth Ravaliya, Prajnya Kalmane, Vishal Chitroda